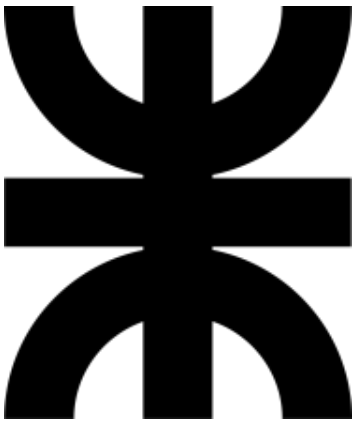
Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información



Trabajo Práctico Evaluable 6 - Test Driven Development

Materia: Ingeniería y Calidad de Software

Año: 2026

Curso: 4K3

Grupo: 2

Integrantes:

- Almeyra, Agustín - 400822
- Ambroggio, María Angélica - 403377
- Chiarini, Antonia - 400532
- Ferreyra, Martin - 404034
- Filipetta, Francisco - 400644
- Fournier, Josefina - 400899
- Germano, Francisco - 403596
- Makarchuk, Maria Eugenia - 402291
- Oviedo, Ignacio - 400169
- Ruffino, Ornella - 400471
- Silberstein, Gaspar - 400137
- Uranga, Samira Anael - 404118

Fecha de Entrega: 21/10/2026

Índice

Desarrollo..... 3
Contexto..... 3
Lenguaje, herramientas y estilo de código..... 3
Arquitectura: dominio separado de la web..... 3
Dependencias externas: tiempo, mail, QR, usuario y datos..... 4
Reglas de negocio y decisiones del grupo..... 5
Estrategia de pruebas y ciclo TDD..... 6
Trazabilidad de las pruebas..... 7
Fuera de alcance..... 8
Anexo..... 8
Términos y condiciones..... 8
Logotipo..... 9
Conclusión..... 10

Desarrollo

Contexto

Se implementa con TDD la User Story "**Inscribirme a actividad**" de EcoHarmony Park, en Python, con pruebas automatizadas en pytest y una interfaz web.

Las fuentes de requerimientos son tres: la US con sus criterios de aceptación y pruebas de usuario, el enunciado del Práctico 6 y el FAQ de los Product Owners. Toda regla de negocio implementada cita su origen. Lo que no está escrito en esas fuentes y tuvimos que definir, está marcado como **decisión del grupo**.

Lenguaje, herramientas y estilo de código

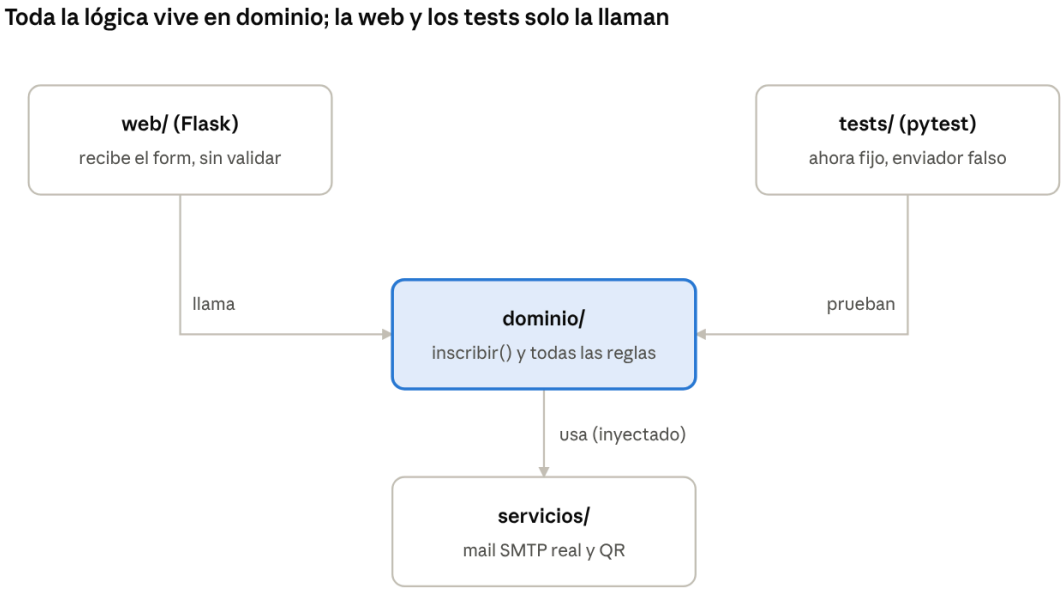
Elemento	Elección	Por qué
Lenguaje	Python 3	Es el lenguaje con más dominio en el grupo
Pruebas	pytest	Tests como funciones simples con `assert`, sin clases ni boilerplate
Interfaz	Flask + HTML/CSS	El FAQ #24 pide una app mobile y admite web responsive. El flujo formulario, POST, función Python se explica en una oración
Datos	SQLite (archivo local) y un seed	SQLite viene con Python y no hay que instalar nada. Los tests de dominio usan una lista y los de persistencia un archivo temporal
QR	Librería `qrcode`	Generación estándar, sin implementar la codificación
Estilo	PEP 8	Convención estándar de Python, verificada con pycodestyle.

Descartamos Tkinter y CustomTkinter porque no permiten una interfaz responsive ni aplicar fácilmente la fuente Lato y la paleta del FAQ #3.

Descartamos Postgres y MySQL porque suman instalación y un punto de falla (servidor, usuario y contraseña) sin aportar nada a lo que se evalúa. SQLite no necesita nada de eso.

Arquitectura: dominio separado de la web

Toda la lógica y las validaciones viven en `dominio/`, en una función `inscribir(...)`. La capa web no valida nada: recibe el formulario, llama a esa función y muestra el resultado o el mensaje de error.



Los tests prueban `dominio/` directamente, sin abrir el navegador, así que corren en milisegundos. Así se cumple la consigna de probar todas las reglas aunque la interfaz ya las bloquee, porque el test valida la regla en el lugar donde vive. Cambiar la interfaz no toca ningún test.

Dependencias externas: tiempo, mail, QR, usuario y datos

Todo lo que el código no controla (el reloj, la red, el usuario) se recibe desde afuera. Así los tests dan siempre el mismo resultado.

- **"Ahora" inyectado:** `inscribir(...)` recibe la fecha y hora actual como parámetro. Los tests usan una fecha fija, la app pasa `datetime.now()`. Si la función leyera el reloj por su cuenta, un test como "turno ya pasado" daría distinto resultado según el día en que se corra.
- **Mail real, pero inyectado:** La app envía mails reales por SMTP (interpretamos el "Desarrollo" del FAQ #5 como implementarlo; se pidió lo mismo para Mercado Pago). `inscribir(...)` recibe el enviador como parámetro: la app usa el enviador SMTP real y los tests un enviador falso que solo registra qué se envió. Así los tests no dependen de internet ni llenan una casilla de mails.
- **Credenciales fuera del repositorio:** El usuario y la contraseña de aplicación SMTP viven en un archivo `.env` excluido por `.gitignore`. Al repositorio público solo se sube `.env.example` con los campos vacíos.
- **Usuario mockeado** (FAQ #19)**:** No hay registro ni login. El mail destino es el del usuario mock, configurado en `.env`.
- **QR: se prueba qué contiene, no la imagen:** Se genera uno por visitante (FAQ #28) con la librería `qrcode`. El texto que codifica se arma en el dominio (`texto_qr`) y está testeado: un QR por visitante, con los datos de la inscripción. La imagen la dibuja la librería: no

probamos la librería, y el código que la llama (servicios/qr.py) se verifica a mano escaneando el QR del mail.

- **Registro inyectado y SQLite:** inscribir recibe el registro de inscripciones y solo le pide agregar, recorrer y contar. Los tests le pasan una lista; la app, un RegistroSQLite que guarda en el archivo eco_harmony.db (fuera de git), con una transacción por inscripción. Cada test arma su propio registro, así ningún test depende de otro. Al arrancar, un seed llena el próximo turno de Safari para la demo y no lo duplica.

Reglas de negocio y decisiones del grupo

Cada regla implementada tiene un origen escrito. Las filas marcadas como decisión del grupo no estaban definidas en ninguna fuente.

Regla	Valor	Origen
Actividades	Tirolesa, Safari, Palestra, Jardinería	US
Cupo por turno	Safari 8, Palestra 12, Jardinería 12, Tirolesa 10	FAQ #13
Turnos	Uno por hora, inicios de 9:00 a 17:00	FAQ #12
Días cerrados	Lunes, 25/12 y 1/1. En pantalla se avisa y se vacía el campo; el dominio igual los rechaza	FAQ #1 y #2
Ventana de reserva	Desde 24 hs antes hasta 15 min antes del turno	FAQ #23 y #30
Talle obligatorio	Palestra y Tirolesa; talles XS a XXL	FAQ #9 y #14
Edad mínima	Palestra 12, Tirolesa 8, resto sin límite	FAQ #15
Personas por inscripción	Sin máximo; el único límite es el cupo	FAQ #31
Superposición	Un DNI no puede estar en dos actividades en el mismo horario	FAQ #29
Términos y condiciones	Uno distinto por actividad	US, FAQ #16
Formato del DNI	Solo dígitos, de 1.000.000 a 99.999.999, sin puntos ni ceros adelante	Decisión del grupo
Falla del mail	Si el mail no se envía, no hay inscripción	Decisión del grupo
Orden de las operaciones	Validar → enviar mail → registrar y descontar cupo	Decisión del grupo

Regla	Valor	Origen
Nombre	Nombre y apellido: 2 o más palabras, solo letras (con tildes, ñ, apóstrofo o guion); sin números ni puntos	Decisión del grupo
Edad	Número entero de 0 a 120	Decisión del grupo
DNI repetido en la inscripción	Se rechaza si dos visitantes tienen el mismo DNI	Decisión del grupo
Talle en actividad que no lo pide	Se ignora	Decisión del grupo
Número de orden	Inscripciones ya registradas + 1; va en el mail	FAQ #18
Cupos visibles	El usuario ve los lugares libres por turno	FAQ #8
Errores	Se muestra el primer error, debajo del campo que lo causó	Decisión del grupo
Orden de las pantallas	Actividad, turno y cantidad antes que los datos de los visitantes; el cupo se valida al continuar y otra vez al confirmar	Decisión del grupo

Formato del DNI. Ninguna fuente lo define. Adoptamos la convención habitual de los sistemas argentinos: solo dígitos, de 7 u 8 caracteres y sin ceros delante (de 1.000.000 a 99.999.999). Un DNI con puntos o 00000000 se rechaza.

Si el mail falla, no hay inscripción. El criterio de aceptación dice que al finalizar *se debe* enviar el correo. Además, el mail lleva el QR de acceso: sin mail no hay QR, y sin QR la inscripción no sirve para entrar. Por eso el mail se envía antes de registrar la inscripción y descontar el cupo. Si se hiciera al revés, una falla de red dejaría el cupo descontado sin inscripción válida.

Estrategia de pruebas y ciclo TDD

Cada test sale de una regla escrita en la US, el enunciado o el FAQ. Además, seguimos dos criterios de clase: probamos toda regla aunque la interfaz ya la bloquee, y cada mensaje de error distinto tiene su propio test.

Ciclo aplicado en cada test:

1. Escribir una prueba que defina el comportamiento esperado.
2. Preparar el código para que la prueba pueda ejecutarse.
3. Ejecutar la prueba y comprobar que falle (Red), porque todavía no está implementada la funcionalidad.
4. Implementar lo mínimo necesario para que la prueba pase.
5. Volver a ejecutar la prueba y toda la suite de tests para verificar que todo funcione (Green)
6. Refactorizar el código, eliminando duplicaciones y mejorando su estructura sin alterar su comportamiento.

Orden de los ciclos: Fuimos de lo más simple a lo más complejo, para que cada test obligue a agregar un poco más de diseño. Esto es camino feliz, datos faltantes, formatos inválidos, reglas por actividad (talle y edad mínima), cantidad y cupo, reglas de tiempo, superposición, mail y QR. La interfaz web se construyó al final, como una capa delgada sin lógica; después se agregó la persistencia con SQLite.

Evidencia en el historial: Cada fase del ciclo es un commit propio (`test: … (red)`, `feat: … (green)`, `refactor: …`), así el repositorio muestra el ciclo Red, Green, Refactor de cada test.

Estructura de cada test: preparar los datos, ejecutar `inscribir(...)` y verificar el resultado (Arrange / Act / Assert), con datos fáciles de leer y sin depender de otros tests.

Trazabilidad de las pruebas

Los 74 tests pasan con `python -m pytest -v`. Los de la US se corresponden con sus pruebas de usuario; el resto cubre criterios de aceptación, el FAQ y decisiones del grupo.

Prueba de usuario de la US	Tests
Inscripción exitosa con mail (pasa)	`test_inscripcion_exitosa`, `test_envia_mail_al_confirmar`
Sin cupo en el horario (falla)	`test_actividad_sin_cupo`
Sin talle porque no se requiere (pasa)	`test_talle_no_requerido_omitido`
Horario cerrado o no disponible (falla)	`test_turno_fuera_de_rango`, `test_dia_lunes_cerrado`, `test_dia_feriado_cerrado`
Sin aceptar términos (falla)	`test_no_acepta_terminos`
Sin talle requerido (falla)	`test_talle_requerido_faltante`

Los demás tests de test_inscripcion.py prueban las reglas de la tabla anterior. Los de test_seed.py y test_registro_sqlite.py (3 y 3) son de soporte: prueban el seed y la persistencia, no la US. Todos siguen el ciclo Red → Green → Refactor, salvo los que pasaron de entrada, que figuran así en el historial y en la bitácora. Seis tests de borde (cupo exacto, turnos de las 9:00 y las 17:00, mismo DNI en otro horario, talle ignorado, DNI en los límites del rango y edad máxima) se agregaron en una revisión posterior, con el código ya hecho: su Red se verificó rompiendo el código a propósito y revirtiéndolo después.

Fuera de alcance

Nos limitamos a lo que piden la US, el enunciado y el FAQ. Quedó afuera:

- **Registro y login:** el FAQ #19 permite trabajar con la sesión y el usuario mockeados.
- **Reglas de "Comprar entradas":** precios, descuentos, pago y límite de 10 entradas son de la otra US.
- **Test de la imagen del QR:** se prueba el texto que codifica, no el dibujo. Tampoco se testea el envío SMTP real. Ambos se verifican a mano al recibir el mail y escanear su QR.
- **Base de datos:** no usamos Postgres ni MySQL; SQLite alcanza para que las inscripciones sobrevivan al reinicio. No se contempla concurrencia alta: dos confirmaciones simultáneas podrían recibir el mismo número de orden. La transacción de guardado no tiene test; se verifica leyendo el código.
- **Documentos extranjeros:** el FAQ #32 admite solo DNI.

Anexo

Términos y condiciones

Tirolesa: *Es obligatorio usar el arnés y el casco que entrega el parque. Hay que seguir en todo momento las indicaciones del instructor. No se puede participar bajo efectos del alcohol, con problemas cardíacos o en estado de embarazo.*

Safari: *Hay que permanecer dentro del vehículo durante todo el recorrido. No está permitido alimentar, tocar ni llamar a los animales. Los menores deben ir acompañados por un adulto responsable.*

Palestra: *Es obligatorio usar el arnés y el calzado adecuado, y escalar siempre con la cuerda asegurada por el instructor. Hay que respetar sus indicaciones y no subir sin autorización. No se puede participar con lesiones que impidan hacer esfuerzo físico.*

Jardinería: *Hay que usar los guantes y las herramientas que entrega el parque y cuidarlas durante la actividad. Las plantas y semillas no se pueden retirar del parque. Los menores deben ir acompañados por un adulto responsable.*

Logotipo



Conclusión

Se implementó con TDD la User Story **"Inscribirme a actividad"**: los tests automatizados pasan, cada fase del ciclo Red, Green y Refactor quedó en su propio *commit* y la aplicación funciona de punta a punta, incluido el mail con el QR. Separar el dominio de la web permitió probar todas las reglas sin abrir el navegador, y recibir desde afuera el reloj, el mail y el registro hizo que los tests den siempre el mismo resultado. Lo que ninguna fuente definía, como el formato del DNI, el orden de las operaciones y la falla del mail, quedó escrito como decisión del grupo. Quedan fuera de alcance, y declarados, el registro y login, la concurrencia alta y las reglas de Comprar entradas.

El **código, los tests y demás documentación** están en el repositorio público https://github.com/martinferreyraa/eco-harmony-g2. La versión entregada es el commit **aeefc48.**